

HTTP-based KV Server

CS744 - Design and Engineering of Computing Systems

Md Noman

Roll No. 25M0795

Autumn 2025

Introduction

This project is a simple key-value store built in C++. It uses an HTTP server to handle requests, an in-memory cache to speed up reads, and reduce the database hits and a PostgreSQL database to store data. The main goal is to show how requests are handled differently depending on whether the value is in cache or needs to be fetched from the database.

1 Project Description:

This project is a simple HTTP-based key-value store written in C++. It has three main parts:

1.1 Server (`main.cpp`)

- Uses the `httplib` library to handle HTTP requests.
- Provides three endpoints:
 - **PUT:** Stores a key-value pair in the database.
 - **GET:** First checks the cache; if the key is found, returns the value from cache, otherwise retrieves it from the database.
 - **DELETE:** Deletes a key-value pair from both cache and database.
- Each request is checked for correctness and returns the proper HTTP status.

1.2 LRU Cache (`cache.hpp` / `cache.cpp`)

- Implemented a small in-memory cache using the Least Recently Used (LRU) policy.
- GET requests first check the cache to reduce database hits.
- Thread-safe using a `mutex` so multiple requests can run at the same time.

1.3 Database Layer (db.hpp / db.cpp)

- It connects to PostgreSQL using libpq.
- Supports storing, retrieving, and deleting key-value pairs.
- Uses proper queries to safely and efficiently interact with the database.

1.4 Load Generator

- The load generator is a multi-threaded program that takes number of client threads as a argument.
- It operates in a closed-loop manner: each thread sends a request, waits for the response, and immediately sends the next request.
- It supports generating different workloads by varying the mix of GET/PUT/DELETE operations.

Execution Paths

- **Cache Hit Path:** GET returns the value from cache if it's already there.
- **Database Hit Path:** GET returns the value from PostgreSQL if not in cache, and then updates the cache.

2 Sample Test

Below is a screenshot of some example requests and responses of the key-value store:

```
nomankanodia@NOMAN-KAPOOR:/mnt/d/DECS Final Project$ curl -X PUT -d "hello world" http://localhost:8080/set/greeting
Stored successfully
nomankanodia@NOMAN-KAPOOR:/mnt/d/DECS Final Project$ curl -X PUT -d "red" http://localhost:8080/set/apple
Stored successfully
nomankanodia@NOMAN-KAPOOR:/mnt/d/DECS Final Project$ curl -X PUT -d "yellow" http://localhost:8080/set/banana
Stored successfully
nomankanodia@NOMAN-KAPOOR:/mnt/d/DECS Final Project$ curl http://localhost:8080/get/greeting
Cache hit: hello world
nomankanodia@NOMAN-KAPOOR:/mnt/d/DECS Final Project$ curl http://localhost:8080/get/apple
Cache hit: red
nomankanodia@NOMAN-KAPOOR:/mnt/d/DECS Final Project$ curl -X DELETE http://localhost:8080/delete/greeting
Deleted successfully
nomankanodia@NOMAN-KAPOOR:/mnt/d/DECS Final Project$ curl -X DELETE http://localhost:8080/delete/banana
Deleted successfully
nomankanodia@NOMAN-KAPOOR:/mnt/d/DECS Final Project$
```

Figure 1: Sample test showing PUT, GET, and DELETE requests.

3 System Architecture

The system consists of an HTTP client interacting with a server that handles requests using an in-memory cache and a PostgreSQL database.

- **GET request:** The server first checks the LRU cache. If the key exists, it returns the value directly. If not, it queries the PostgreSQL database and updates the cache.
- **PUT/DELETE request:** The server updates both the cache and the database to ensure consistency.

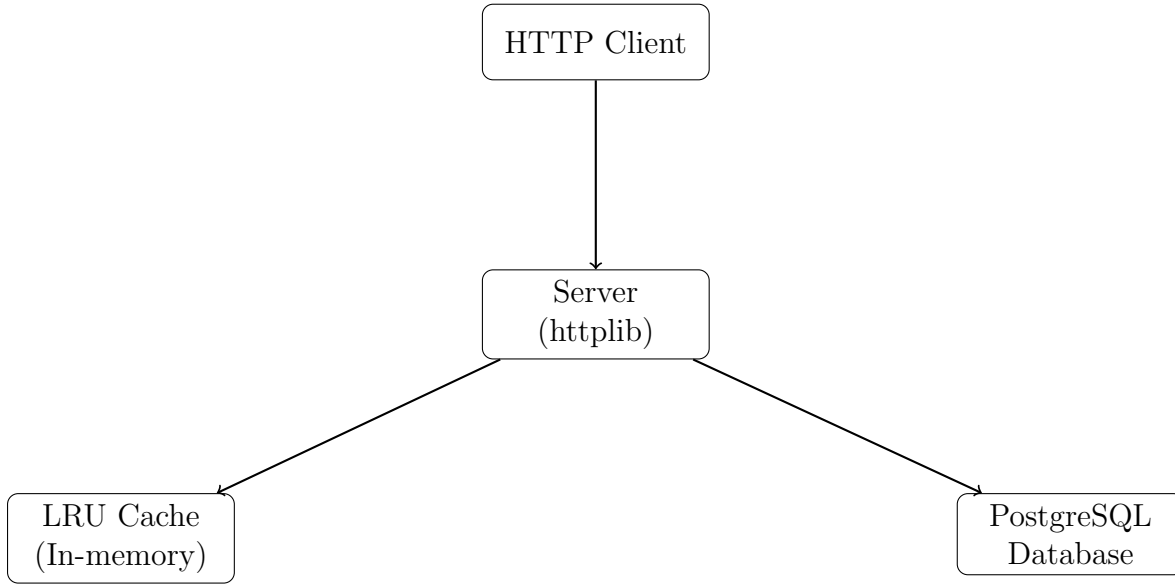


Figure 2: System architecture showing the HTTP client, server, in-memory cache, and database.

4 Load Test Setup

4.1 Hardware and Software Environment

- I used WSL (Windows Subsystem for Linux) to run the server environment.
- I used PostgreSQL with the connection pool of 8 connections as the database to store the key-value pairs.
- Used 6 core divided into server and loadgenerator to perform the load test.

4.2 Running the Server and Database

- The Server and Database were pinned to CPU cores 0-2 using:

```
taskset -c 0-3 ./server
```

- The Load Generator was pinned to CPU cores 3-5 using:

```
taskset -c 4-7 ./load_gen
```

4.3 Workloads

Workload 1:- put_all

- This load test consists of 100% PUT operations .
- Keys chosen from a uniform random distribution of 10,000 possible keys.
- Expected bottleneck: Disk I/O on the database.

Workload 2: get_popular

- This workload is consists of 100% GET operations (Cache bound).
- Keys chosen from a small set of 10 hot keys.
- Expected bottleneck: Server CPU due to a high cache hit rate.

4.4 Load Test Parameters

I varied the number of concurrent client threads like 2, 4, 6, 8..... Each test run for the duration of 30s. I measured the following metrics:

- Throughput (requests/sec)
- Average Response Time (ms)
- CPU Utilization (%)

5 Load Test Results

5.1 Workload 1: get_popular

- **Average Throughput vs. Number of Threads**

- Throughput increases with the increasing number of threads and flatten after it.
- The throughput reached at a much higher rate of approximately 12,000 requests/sec, as most requests are served from the cache.

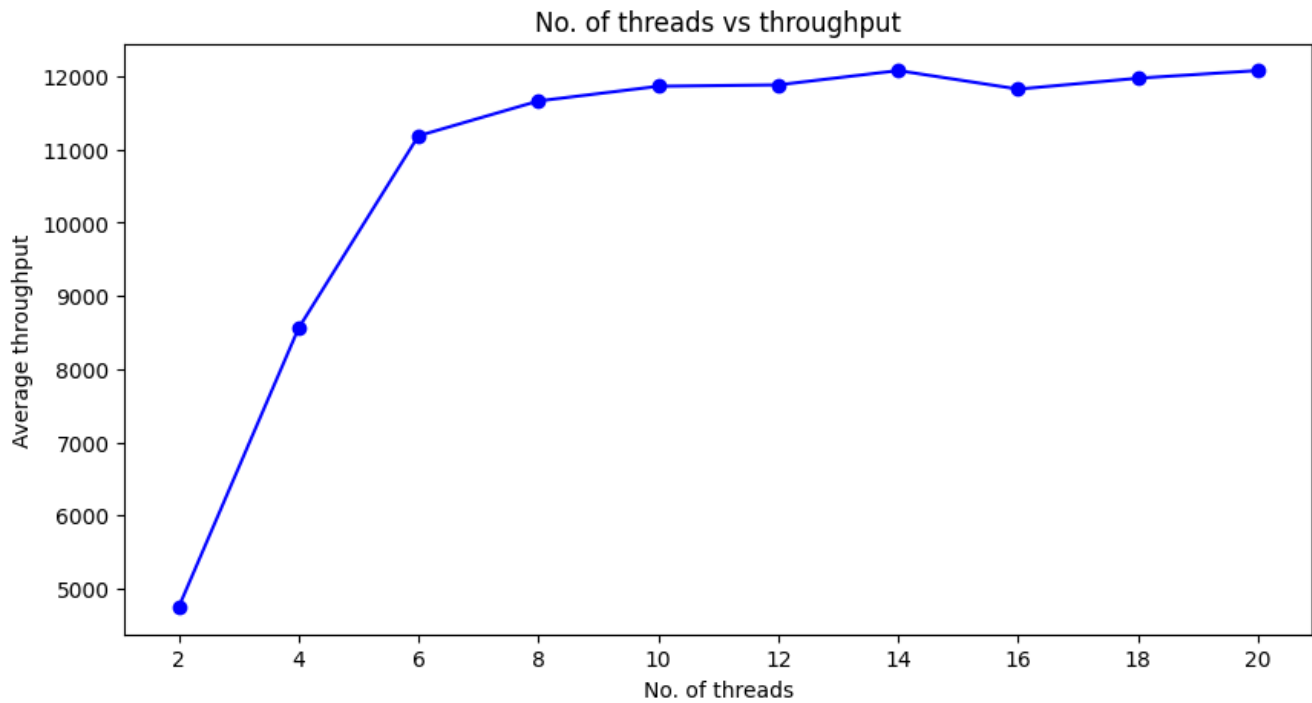


Figure 3: No. of threads vs throughput

- **Average Response Time vs. Number of Threads**

Response time remains low for the less number of threads but increase with the number of threads due to locks on the cache and OS scheduling overhead.

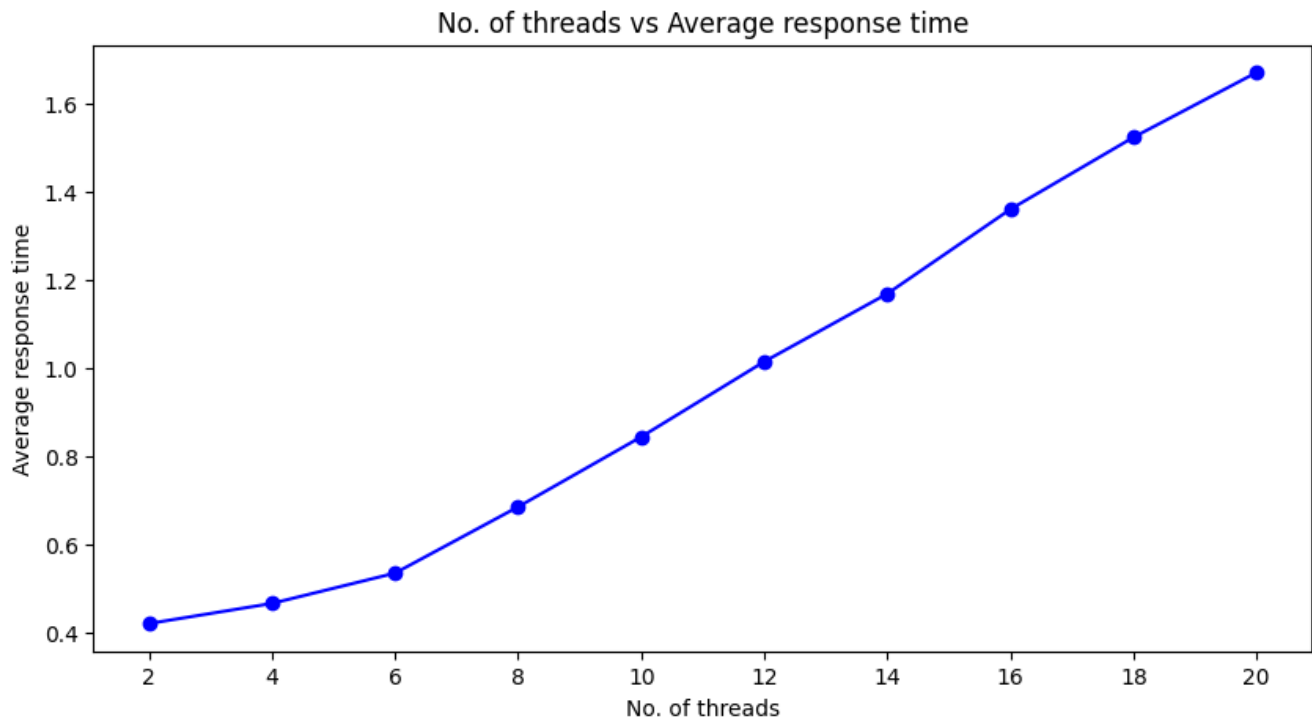


Figure 4: No.of threads vs Average Response time

- **CPU Utilization vs. Number of Threads**

What the Graph Shows:

- The server's CPU utilization reaches 220%.
- After this remains almost same.

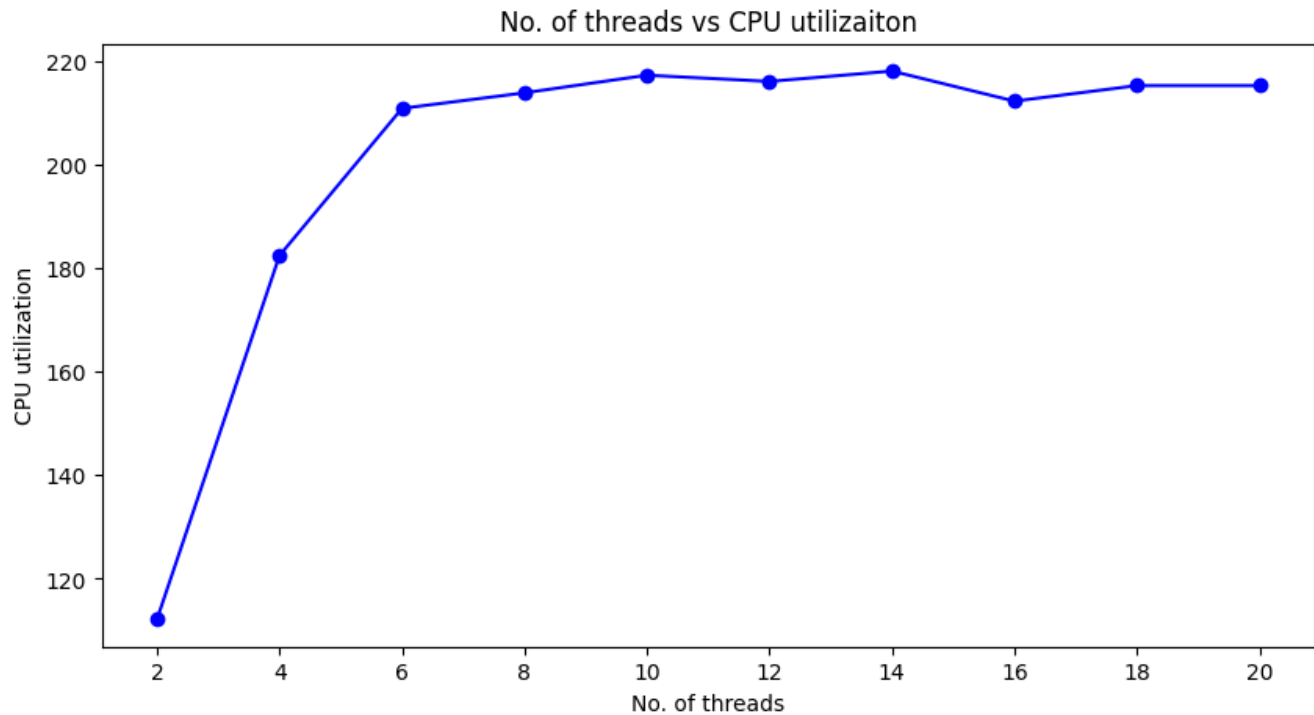


Figure 5: No. of threads vs CPU utilization

- **Bottleneck Analysis:**

The server's CPU cores are fully saturated at 220% utilization, while the database disk is largely idle. This clearly shows that the server's CPU is the primary bottleneck for this cache-heavy, workload.

5.2 Workload 2: put_all

- **Average Throughput vs. Number of Threads**

Throughput increases with the increasing number of threads and flatten after it.

It reached at a much higher rate of approximately 2,400 requests/sec, as most requests are served from the cache.

This indicates the server has reached at its maximum capacity for this workload test.

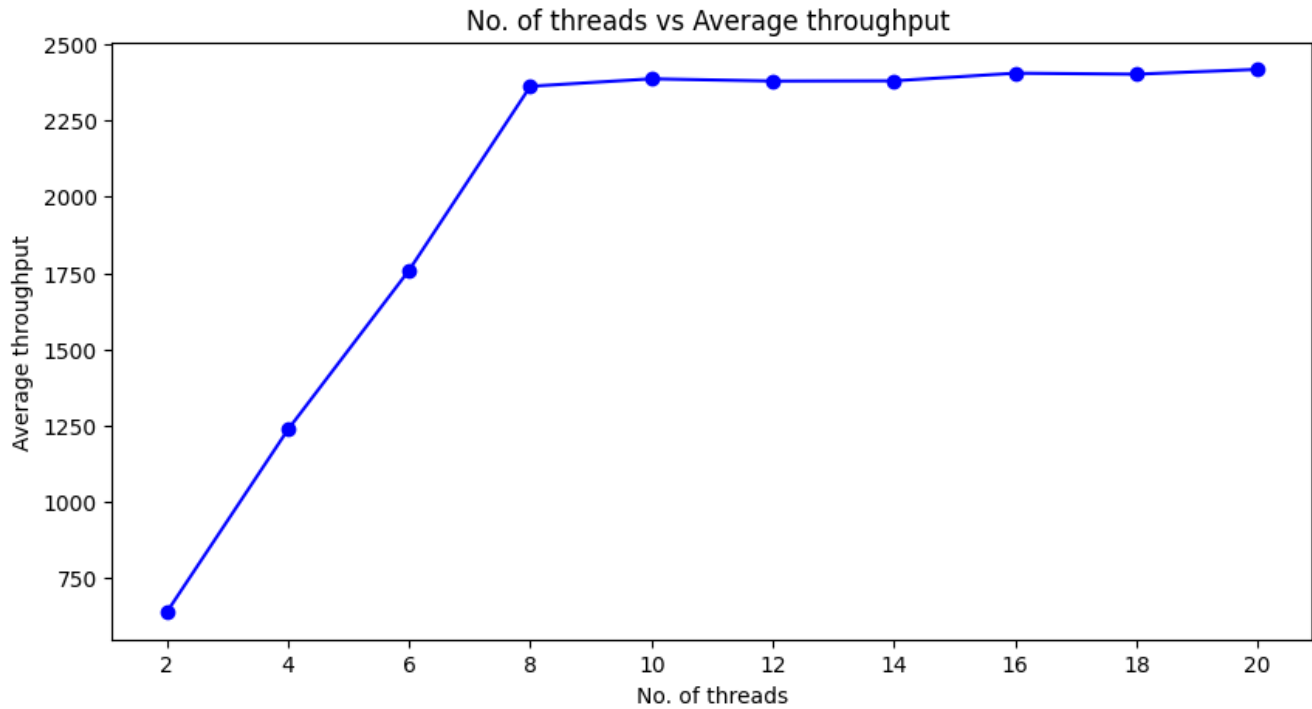


Figure 6: No. of threads vs throughput

- **Average Response Time vs. Number of Threads**

Response time remains low for the less number of threads but increase with the number of threads due to OS scheduling overhead.

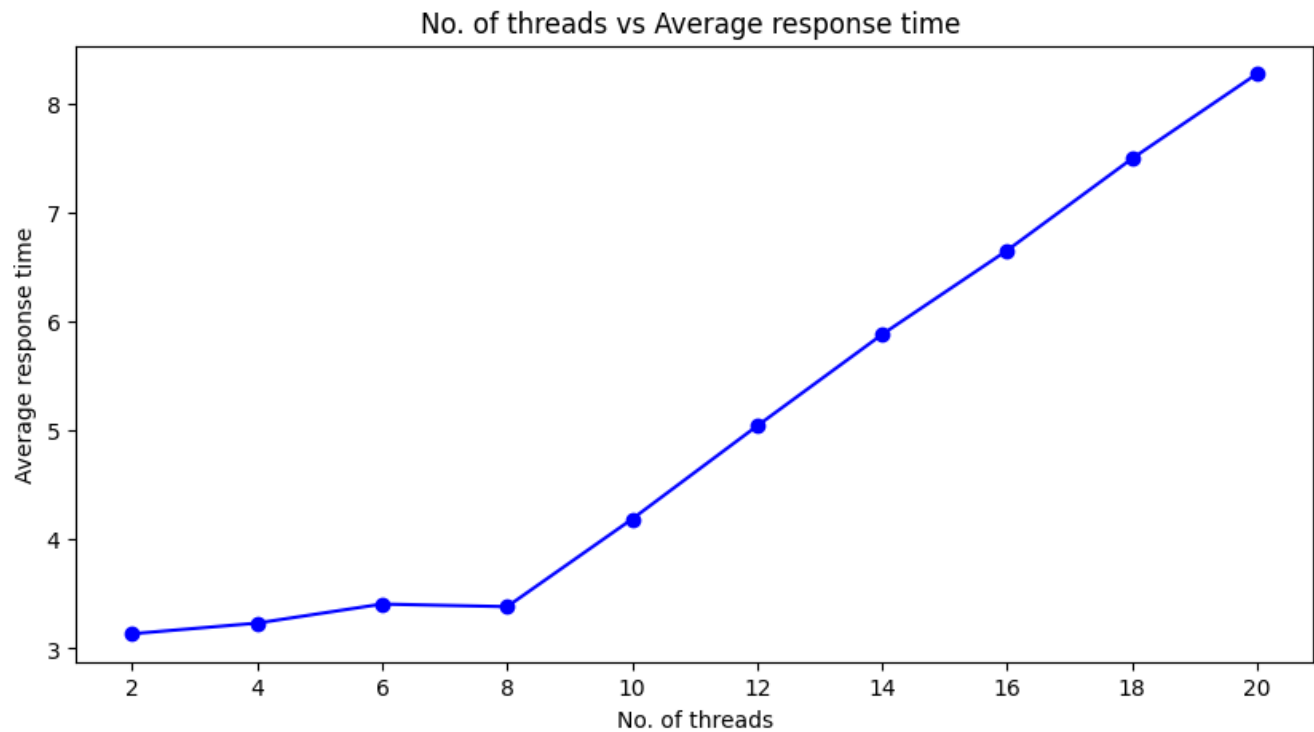


Figure 7: No. of threads vs Average response time

- **CPU Utilization vs. Number of Threads**

What the Graph Shows:

- The server's CPU utilization reaches only at 100% - 110%.
- After this remains almost same.

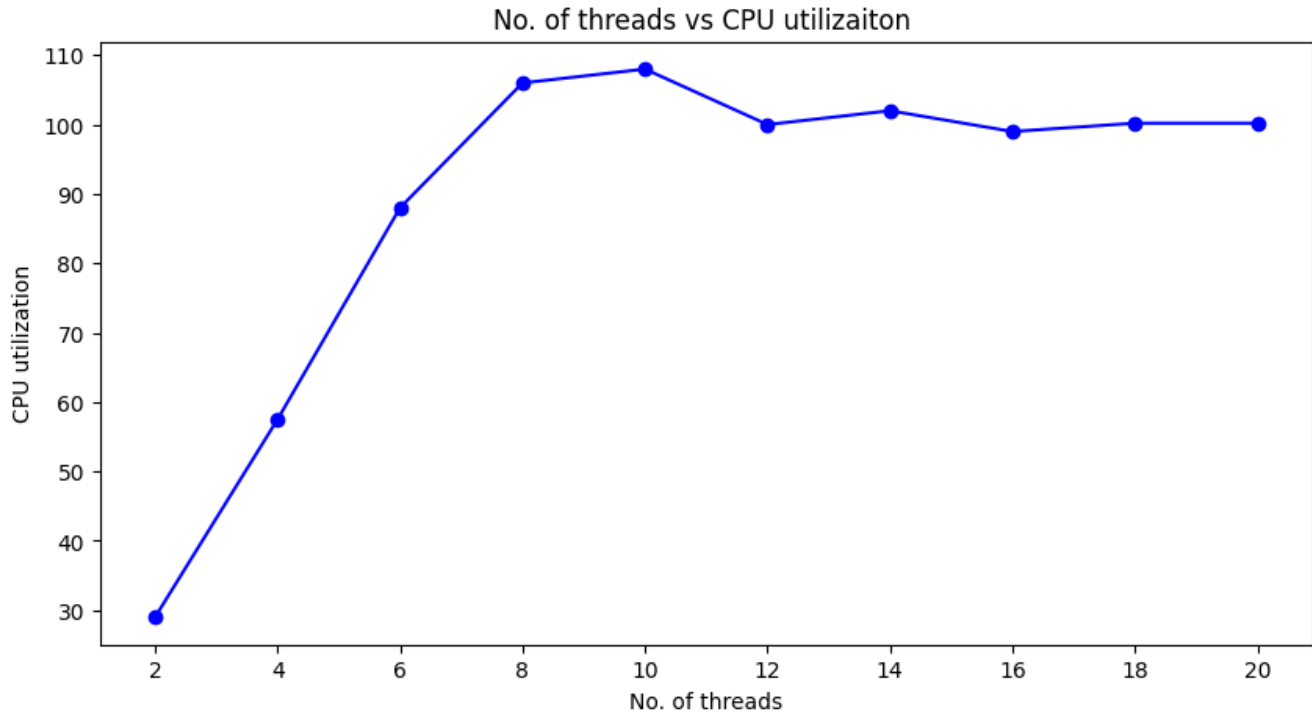


Figure 8: No. of threads vs CPU utilization

- **Bottleneck Analysis:**

The server's CPU cores are remain around only 110% utilization. This clearly shows that the disk I/O is the bottleneck of this workload.

6 Project Repository

Github Repository Link: <https://github.com/NomanKanodia/decs-project>